HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 11

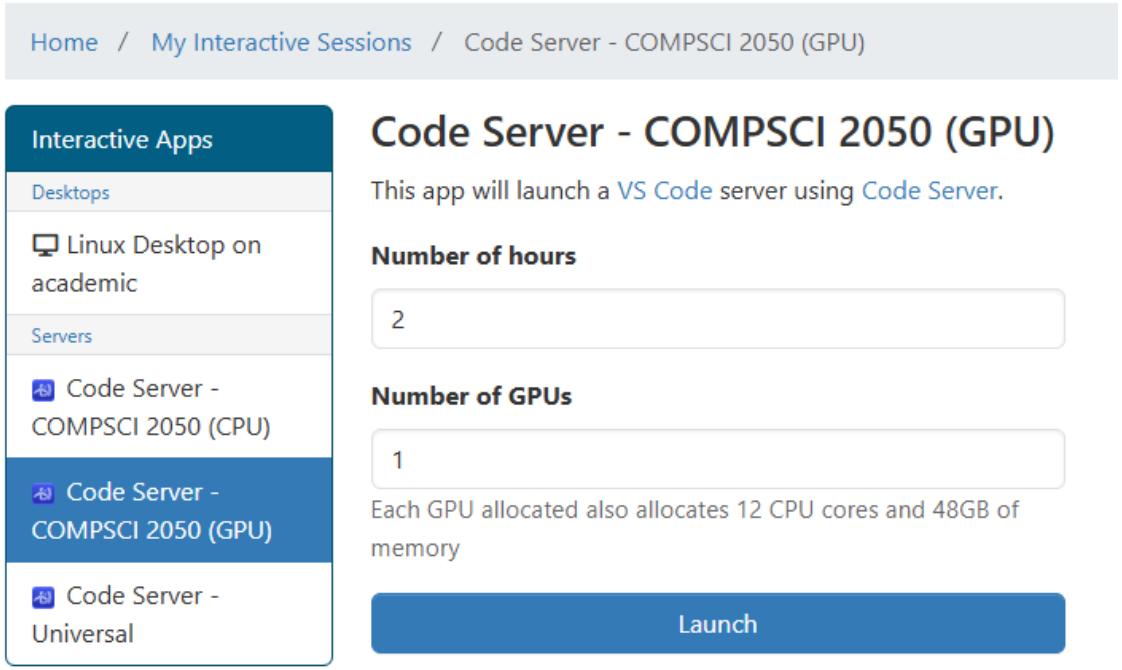
March 3, 2026

SETUP

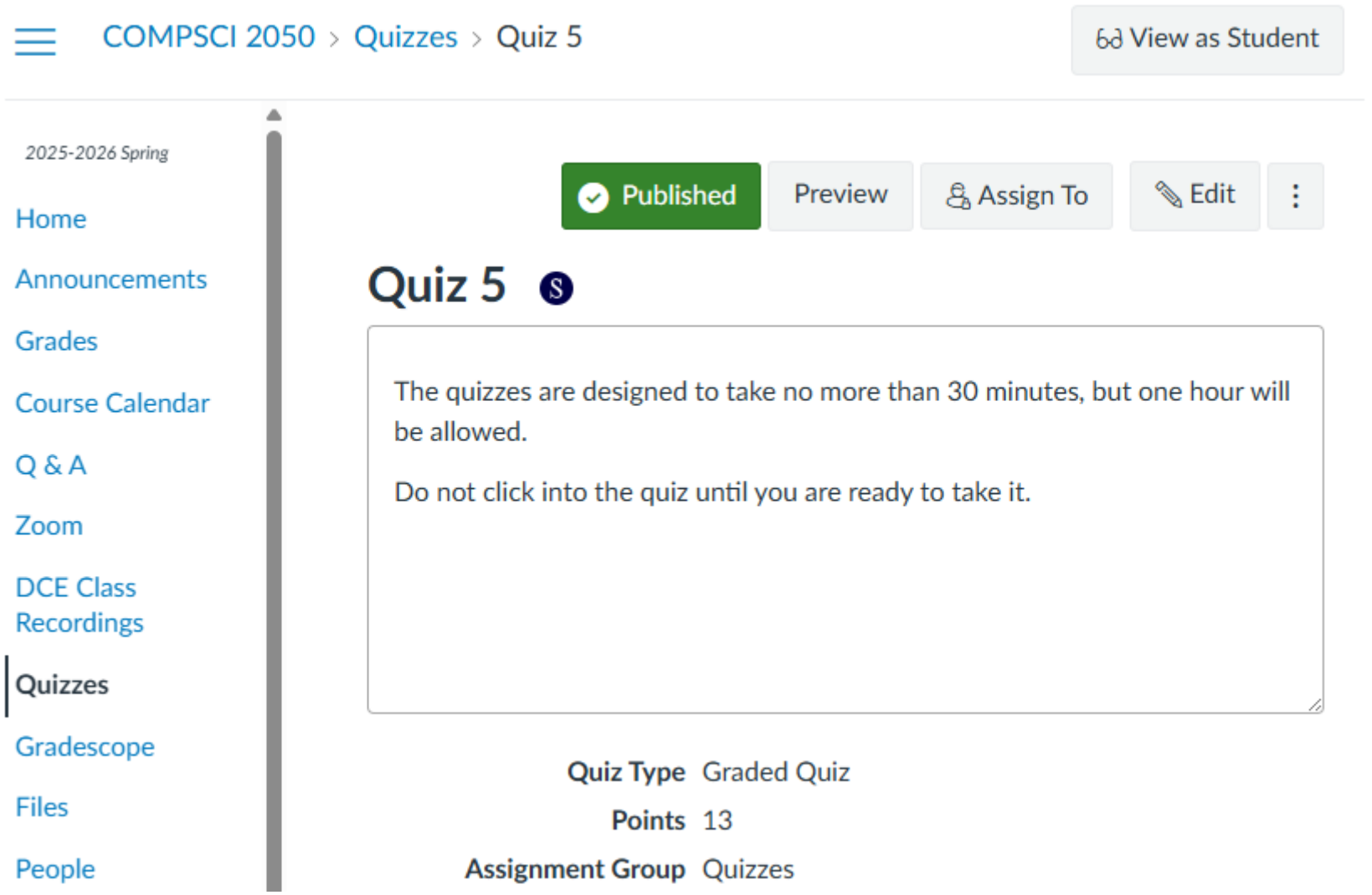
- Submit the `slurm` scripts in example-1 and example-2

```
sbatch submit.sh
```

- Request a GPU



QUIZ 5


COMPSCI 2050 › Quizzes › Quiz 5
View as Student
2025-2026 Spring
Home
Announcements
Grades
Course Calendar
Q & A
Zoom
DCE Class Recordings
Quizzes
Gradescope
Files
People
Published
Preview
Assign To
Edit
Quiz 5
The quizzes are designed to take no more than 30 minutes, but one hour will be allowed.
Do not click into the quiz until you are ready to take it.
Quiz Type Graded Quiz
Points 13
Assignment Group Quizzes

homework-1

Internal template

Edit Pins | Watch 0 | Fork 0 | Star 1 | Use this template

solution | 2 Branches | 0 Tags | Go to file t | Add file | Code

This branch is 1 commit ahead of `main`. | Contribute

Chuck Witt add solution. 65293f9 · 3 days ago | 2 Commits

question-1	public release.	last month
question-2	add solution.	3 days ago
question-3	add solution.	3 days ago
question-4	add solution.	3 days ago
question-5	public release.	last month
question-6	add solution.	3 days ago
question-7	add solution.	3 days ago
README.md	add solution.	3 days ago

README

Homework 1 Solution

You may find it useful to compare this solution with the original assignment.

About

No description, website, or topics provided.

Readme
Activity
Custom properties
1 star
0 watching
0 forks

Releases

No releases published
Create a new release

Contributors 2

wcw398 Chuck Witt
laz380 Laura Zichi

Languages

Python 46.8% C++ 44.6%
CMake 8.6%

CS-2050 / homework-3

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

homework-3

Internal template

Edit Pins | Watch 0 | Fork 0 | Star 0 | Use this template

main | 1 Branch | 0 Tags | Go to file t | Add file | Code

laz380 and Chuck Witt public release. 35621a7 · 4 days ago 1 Commits

question-1	public release.	4 days ago
question-2	public release.	4 days ago
question-3	public release.	4 days ago
question-4	public release.	4 days ago
question-5	public release.	4 days ago
question-6	public release.	4 days ago
question-7	public release.	4 days ago
README.md	public release.	4 days ago

README

Homework 3

Welcome to Homework 3. This assignment is due by 11:59pm on **Thursday March 12.** Begin with the setup instructions in the next section, and don't forget the submission instructions at the bottom.

About

No description, website, or topics provided.

Readme
Activity
Custom properties
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Contributors 2

wcw398 Chuck Witt
laz380 Laura Zichi

Languages

C++ 78.9% Cuda 14.5% Shell 6.0% CMake 0.6%

CS-2050 / questions-and-answers

Type / to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

questions-and-answers

Public

Edit Pins | Watch 0 | Fork 0 | Star 0

main | Go to file t | Code

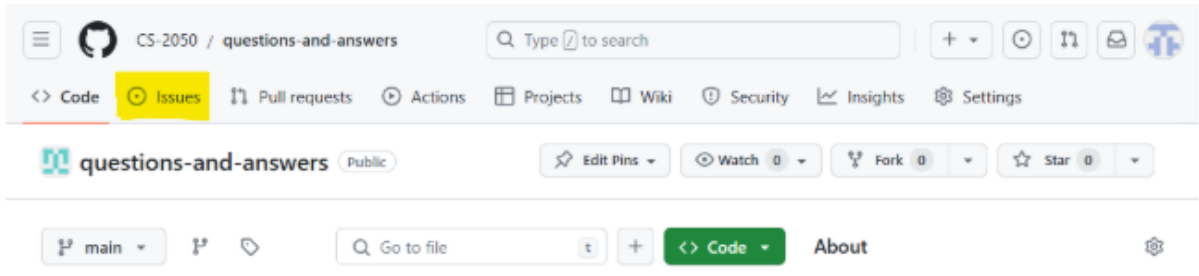
wcw398 update readme 4be70a2 · 17 hours ago 4 Commits

README.md update readme 17 hours ago

README

Questions and Answers

Please create issues in this repository to ask questions about the course.



About

Ask your CS 2050 questions here.

sites.google.com/g.harvard.edu/cs-2050

Readme

Activity

Custom properties

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Closed **submit.slurm fails with "CPU binding outside of job step allocation" on dynamic nodes && srun timing out** #53

mia330 opened this issue last week · 8 comments

```
done

echo -e "\n\n ATOMIC \n\n"
for k in "${threads[@]}"; do
    srun -n 1 -c $k   ./build/atomic
done
```

wcw398 commented 5 days ago

Thanks again for these details! I've now done a deep dive.

The issue is that the first Slurm job (the one that puts you on the node) sets `SLURM_CPU_BIND` to something like

```
SLURM_CPU_BIND=quiet,mask_cpu:0x001000000000
```

When the second Slurm job executes, the previous environment is forwarded to the new machine. Most Slurm variables are reset by Slurm. However, since the second Slurm setup is different, some variables deemed unimportant are left untouched, including `SLURM_CPU_BIND`, which still refers back to the previous job.

I've confirmed that the `--export=NONE` *can* be used to fix this (by disabling environment forwarding), but it's more difficult than I hoped because *nothing* is forwarded - including important PATH and Spack information, which has to be added back manually.

So, we now know why your solution works (because it overrides the outdated `SLURM_CPU_BIND`). But unfortunately, none of the other fixes I can imagine are very elegant, so the recommendation for now is still: "always submit batch jobs from the login node".

HOMEWORK 1, QUESTION 5

CROSS PRODUCT

```
# Create two 3D vectors
num_elements = 3
v1 = np.random.random(num_elements)
v2 = np.random.random(num_elements)

# Function for computing cross(v1, v2) using Python
arithmetic
def cross_py():
    return np.array([
        v1[1] * v2[2] - v1[2] * v2[1],
        v1[2] * v2[0] - v1[0] * v2[2],
        v1[0] * v2[1] - v1[1] * v2[0],
    ])

# Function for computing cross(v1, v2) using the C++
code
def cross_cpp():
    return example.cross(v1, v2)

# Function for computing cross(v1, v2) using
numpy.cross
def cross_numpy():
    return np.cross(v1, v2)
```

“case where numpy is actually quite a bit slower because of the overhead of handling a lot more cases”

```
Python cross product takes:    2.448 seconds.
Numpy cross product takes:    29.712 seconds.
C++ cross product takes:       0.837 seconds.
```

GENETIC ALGORITHM FOR RASTRIGIN

Rastrigin function

4 languages

Article Talk | Read Edit View history Tools

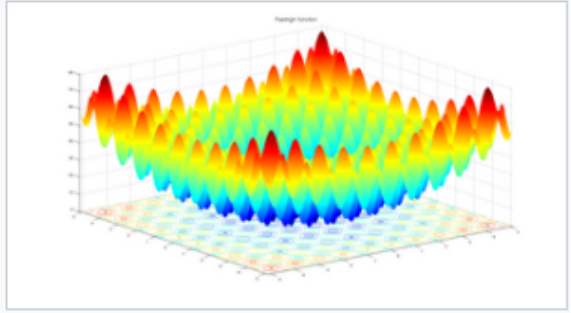
From Wikipedia, the free encyclopedia

In mathematical optimization, the **Rastrigin function** is a non-convex function used as a performance test problem for optimization algorithms. It is a typical example of non-linear multimodal function. It was first proposed in 1974 by Rastrigin[1] as a 2-dimensional function and has been generalized by Rudolph.[2] The generalized version was popularized by Hoffmeister & Bäck[3] and Mühlenbein et al.[4] Finding the minimum of this function is a fairly difficult problem due to its large search space and its large number of local minima.

On an n-dimensional domain it is defined by:

$$f(\mathbf{x}) = An + \sum_{i=1}^{n} \left[x_i^2 - A\cos(2\pi x_i)\right]$$

Rastrigin function of two variables



In 3D

"I work with genetic algorithms-GA/programming-GP in my day job"

~20x speedup

CONWAY'S GAME OF LIFE

Conway's Game of Life

37 languages

Article Talk Read Edit View history Tools

From Wikipedia, the free encyclopedia

"Conway game" redirects here. For Conway's surreal number game theory, see Surreal number.

The **Game of Life**, also known as **Conway's Game of Life** or simply **Life**, is a cellular automaton devised by the British mathematician John Horton Conway in 1970.[1] It is a zero-player game,[2][3] meaning that its evolution is determined by its initial state, requiring no further input. One interacts with the Game of Life by creating an initial configuration and observing how it evolves in discrete time steps called generations. There is no limit to the number of generations (computational resources permitting) or win condition.

A single Gosper's glider gun creating gliders

Rules [edit]

The universe of the Game of Life is an infinite, two-dimensional orthogonal grid of square *cells*, each of which is in one of two possible states, *live* or *dead* (or

"What is more interesting than "life" itself?!"

"Pybind11 was incredibly easy to use."

~50-500x speedup

MONTE CARLO PRICER FOR A CALL OPTION USING BLACK-SCHOLES

```
For 5 evals, Python MC takes:  2.957 seconds.
For 5 evals, C++ MC takes:     0.054 seconds.
Speedup (Python/C++): 54.56x
```

I implemented a Monte Carlo pricer for a European call option under geometric Brownian motion (GBM). For each simulated path, the terminal price is

$$S_T = S_0 \exp\left(\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}, Z\right),$$

and the payoff is:

$$\max(S_T - K, 0).$$

I also compute Delta using the pathwise derivative. For a call option:

$$\Delta = e^{-rT}, \mathbb{E}\left[\mathbf{1}_{S_T > K} \cdot \left(\frac{S_T}{S_0}\right)\right].$$

The function returns `(price, delta, standard_error)`.

This is a fun example to compare a large number of independent simulations doing heavy scalar math inside a tight loop. It's compute-dominated (exp/log/sqrt) and embarrassingly parallel across paths, so it's a good candidate for native optimization and parallelization.

The difficult part was figuring out how to make Python and C++ comparable. I used the same deterministic, counter-based RNG in both languages (splitmix64(seed, i)) and the same Box–Muller transform to generate normals. This avoids shared RNG state and makes the algorithm parallel-safe. The Python script verifies that price, delta, and standard error match to a tight numerical tolerance.

BRUTE FORCE ATTEMPT TO GUESS A PASSWORD

My love for programming was born of curiosity about cybersecurity! I have always been fascinated with password "recovery" programs like John the Ripper, THC Hydra, Cain & Able, and Ophcrack, but have never made a serious attempt at building one myself.

The example I produced for this problem uses basic crypto boilerplate code to hash an initial random 4-character password using SHA256, then performs a brute-force password attack, producing each possible combination of alphanumeric characters then hashing them to compare against the target hash. If we find a match, we have found the original plaintext password.

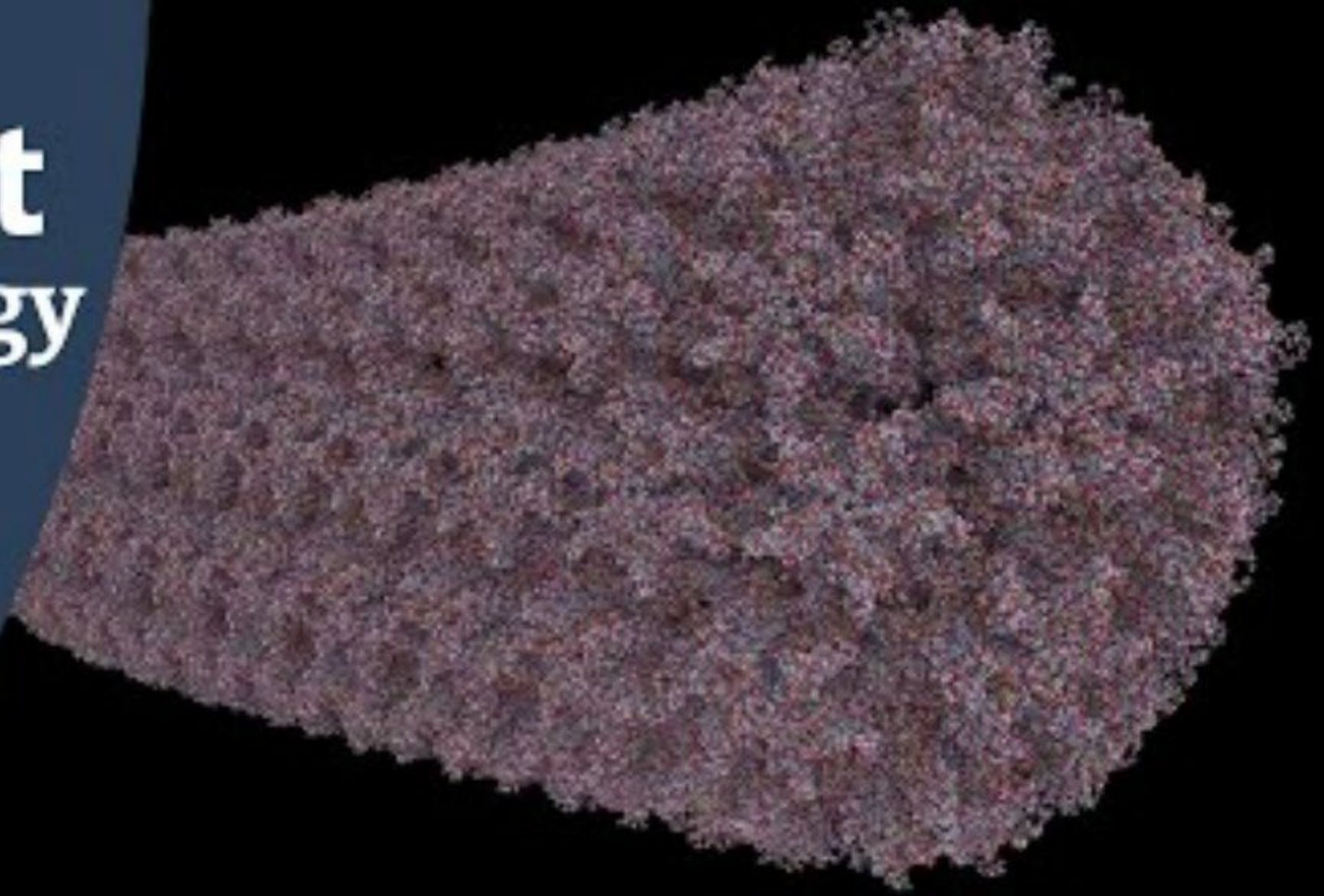
The password guessing code is completely unoptimized which I think makes a great comparison of Python interpreted code versus functionally similar compiled native C++ code. It could potentially make a really fun course project topic - I can already see where parallelization and fine-tuning could provide huge performance improvements when searching for matches. For example, the current algorithm finds a match quite quickly when the target password starts with a character early in the alphabet, but later characters compound into more slowness. It could be beneficial to spawn multiple threads trying substrings for each letter, perhaps even spawning a tree of threads for evaluating the different levels as the number of chars increases. Granted, pre-computed hashes with O(1) lookup are hard to beat for shorter password lengths, but it is a fun problem space to explore.

Interestingly, although the C++ code performs noticeably better than the Python, it's not by as large a factor as I expected. This could be explained by the fact that the hashing algorithm uses similar libraries under the hood for both implementations, and the Python library likely uses native code as well for performance. The hardest part about this exercise was to put together boilerplate hash functions that are somewhat legible without complicated dependencies (I'm far from a crypto expert and had to use examples).

EXAMPLE 1

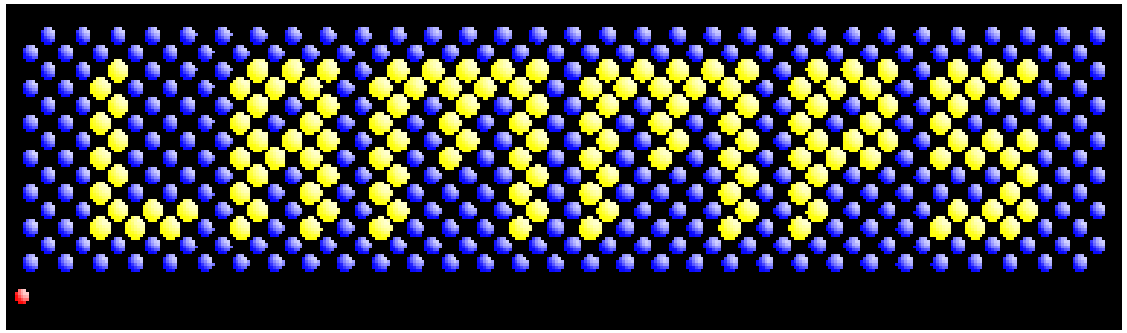
PROFILING WITH VTUNE (THE BASICS)

New Scientist Technology



LAMMPS

Large-scale Atomic/Molecular Massively Parallel Simulator



https://github.com/lammps/lammps

Have a quick look at the codebase.

Get the Intel® VTune™ Profiler

Operating System

Linux* **Windows***

Installation/Package Type

Offline Installer

Online Installer

Intel® VTune™ Profiler for oneAPI: Offline Installer

Find and fix performance bottlenecks quickly and realize all the value of your hardware.

For the most current functional and security features, update to the latest version as it becomes available.

Size	732.37 MB
Version	2025.9.0
Date	February 18, 2026
SHA384	ec7f0fc89435bb7de45db42bb614628f e2ac8fc219c3de7b74844f6387f52972 aa89612f06cf9cd79f45a20558c65081

Intel® VTune(TM) Profiler (version 2025.9.0) has been updated to include functional and security updates. Users should update to the latest version.

EXAMPLE 2

GETTING TO KNOW THE GPU NODES

CUDA REVISITED

CUDA

- ***CUDA:***

  C*ompute* ***U****nified* ***D****evice* ***A****rchitecture* is a programming model for GP GPU programming with support for C/C++/Fortran programming languages. CUDA ships with a large API for host/device coordination.
- *Host* denotes the architecture that hosts the GPU, commonly a CPU.
- *Device* denotes the GPU.
- A *kernel* is a CUDA program (function) that is executed *on each CUDA thread*.
- *CUDA threads* are software threads that each execute the same kernel function.
- A *warp* is a collection of 32 CUDA threads executed together (similar to SIMD width). Warps are scheduled for execution on a *streaming multiprocessor* similar to a physical thread.

CUDA: VECTOR ADDITION

Consider the operation: $y = y + x$ **with** $x, y \in \Re^n$

C++ kernel:

```
1 void add(int n, float *x, float *y)
2 {
3     for (int i = 0; i < n; i++)
4         y[i] = x[i] + y[i];
5 }
```

CUDA kernel (naive):

```
1 __global__ void add(int n, float *x, float *y)
2 {
3     for (int i = 0; i < n; i++)
4         y[i] = x[i] + y[i];
5 }
```

Function execution space specifiers:

- `__global__`: declares a function to be a *GPU kernel*.
- `__device__`: declares a function that executes on the GPU and *is only callable on the GPU*.
- `__host__`: declares a function that executes on the host and *is callable on the host only*.
- The `__device__` and `__host__` specifiers can be used together. Two object files will be generated in this case.
- A `__global__` function must have a `void` return type and cannot be a member of a class. A call to a `__global__` function is *asynchronous* (it returns before the GPU has completed the task) and the call must specify its *execution configuration*.

CUDA: VECTOR ADDITION

```
1 int main(void) {
2     int N = 1 << 20; // 1048576 elements
3     float *x, *y;
4     // Unified Memory: accessible from CPU or GPU
5     cudaMallocManaged(&x, N * sizeof(float));
6     cudaMallocManaged(&y, N * sizeof(float));
7     // Initialize x and y arrays on the host
8     for (int i = 0; i < N; i++) {
9         x[i] = 1.0f;
10        y[i] = 2.0f;
11    }
12    // Run kernel on 1M elements on the GPU (1 thread block of 1 thread)
13    add<<<1, 1>>>(x, y);
14    // Wait for GPU to finish before accessing on host
15    cudaDeviceSynchronize();
16    // Free memory
17    cudaFree(x);
18    cudaFree(y);
19    return 0;
20}
```

Steps

- Allocate unified memory (accessible from host or device).
- Initialize the data on the host (accessing unified memory).
- Launch the GPU kernel. A kernel call has the following general execution configuration (often only the first two parameter within the triple-chevrons are required):

```
f<<<GridDim, BlockDim, ShMem, Stream>>>(param list);
```

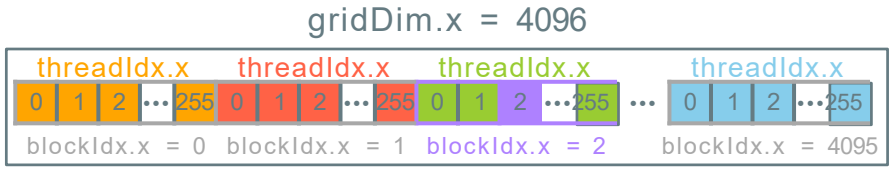
- Synchronize the GPU with the caller on the host (kernel calls are *asynchronous*).
- Free the allocated resources.

CUDA: GRIDS, BLOCKS AND THREADS

- CUDA kernels are launched on a *grid of thread blocks*.
- In each kernel you have access to the execution configuration with *built-in variables*:
  - `threadIdx`: the thread index within a thread block.
  - `blockIdx`: the block index in the kernel launch grid.
  - `blockDim`: the thread block dimensions (number of threads in each dimension).
  - `gridDim`: the dimension of the launch grid (number of thread blocks in each dimension).

Each of these built-in variables are 3D.
For example: `threadIdx.x`, `threadIdx.y` and `threadIdx.z` → CUDA supports 3D grids at most.

1D kernel grid:



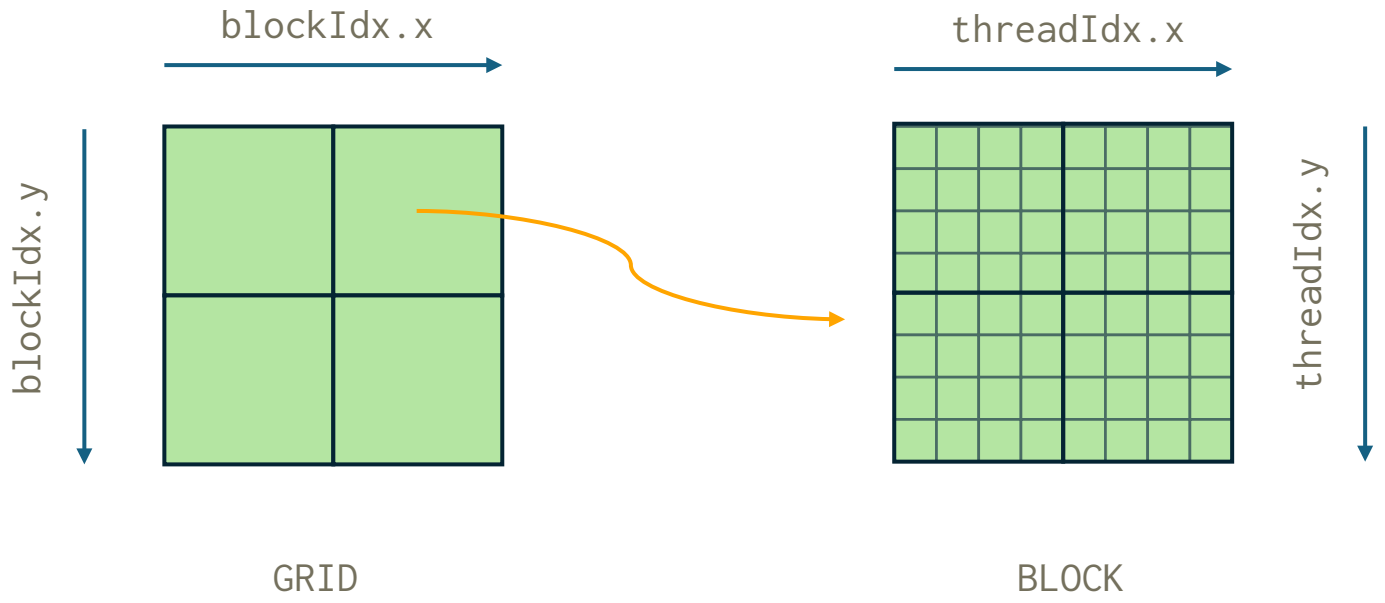
`tid = blockIdx.x * blockDim.x + threadIdx.x`

Example: `tid = 2 * 256 + 2 = 514`

```
1  int main(void)
2  {
3      int N = 1 << 20; // = 4096 * 256
4      // skipping other code
5
6      // Run kernel on 1M elements on the GPU (with TLP)
7      int block_size = 256;
8      int n_blocks = (N + block_size - 1)  / block_size;
9      add<<<n_blocks, block_size>>>(N, x,  y);
10
11     // skipping other code
12     return 0;
13 }
```

CUDA: GRIDS, BLOCKS AND THREADS

- The number of grid dimensions can be up to $2^{31}-1$ on the x axis, 2^{16} for y and z.
- The numbers of threads per block can be up to 1024.



CUDA: WARPS

- Each thread is identified by a unique index `threadID`.
- For 1D grids, it is computed as

```
threadID = threadIdx.x + blockDim.x * blockIdx.x
```

- For 2D grids,

```
threadID = BlockNumInGrid * threadsPerBlock + threadNumInBlock

threadsPerBlock   = blockDim.x * blockDim.y
threadNumInBlock  = threadIdx.x + blockDim.x * threadIdx.y
blockNumInGrid    = blockIdx.x + gridDim.x * blockIdx.y
```

CUDA: WARPS

- Each thread is identified by a unique index threadID.
- Each 32 consecutive `threadIDs` are grouped in a warp.

- All the threads of the same warp are executed by the thread executor.
- Sequential memory operations of the same warp are executed as a single operation.
- This is known as **global memory coalescing**.

EXAMPLE 3

FIRST CUDA KERNELS